



មហាវិទ្យាល័យវិស្វកម្ម  
FACULTY OF ENGINEERING

# Computer Programming

## Lecture 3 Values and Variables - 1

Chhoeum Vantha, Ph.D.  
Telecom & Electronic Engineering

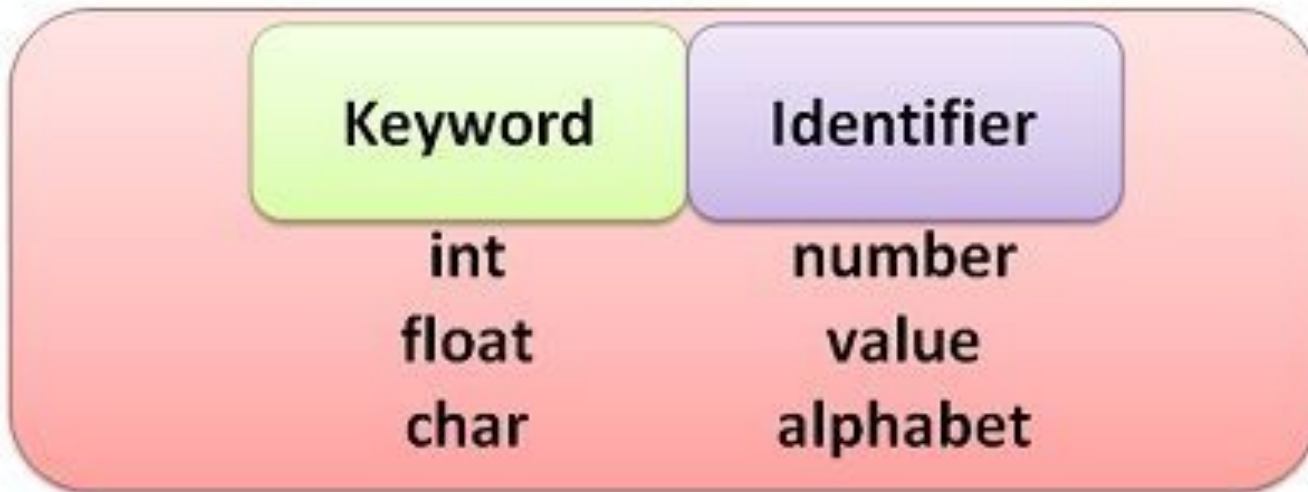
# Contents

- Identifiers
- Variables and Data Type
- Operations
- Assignment Statements and Assignment Expressions
- Named Constants

# Identifiers

- **Identifiers** are the names that identify elements such as **variables, functions, classes, and objects** in a program.

## Variable



# Identifiers

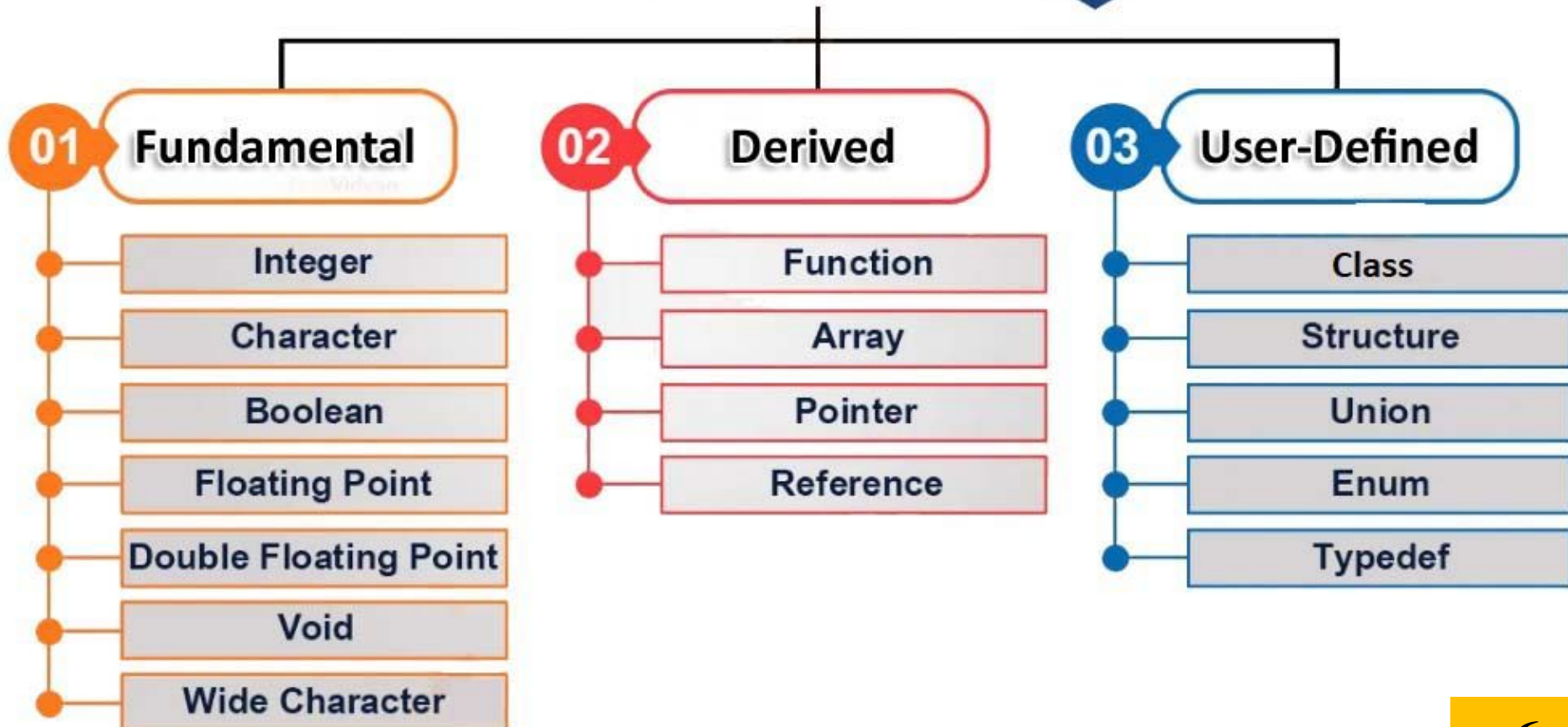
- An identifier is a sequence of characters comprising letters, digits, and underscores (\_).
- An identifier must start with a letter or an underscore; it cannot start with a digit.
- An identifier cannot be a reserved word.
- An identifier can be of any length, but your C++ compiler may impose restrictions.
- Use identifiers of 31 characters or fewer to ensure portability.

# Variables

- Represent data of a certain type.
- To use a variable, you declare it by telling the compiler its name as well as the type of data it can store.
- The variable declaration tells the compiler to allocate appropriate memory space for the variable based on its data type.

# Variables and Data Type

## Data Type in C++



# Variables and Data Type

- Data type and size

| Name           | Synonymy           | Range                                                                                                                      | Storage Size    |
|----------------|--------------------|----------------------------------------------------------------------------------------------------------------------------|-----------------|
| short          | short int          | $-2^{15}$ to $2^{15}-1$ ( $-32,768$ to $32,767$ )                                                                          | 16-bit signed   |
| unsigned short | unsigned short int | 0 to $2^{16}-1$ (65535)                                                                                                    | 16-bit unsigned |
| int signed     |                    | $-2^{31}$ to $2^{31}-1$ ( $-2147483648$ to $2147483647$ )                                                                  | 32-bit          |
| unsigned       | unsigned int       | 0 to $2^{32}-1$ (4294967295)                                                                                               | 32-bit unsigned |
| long           | long int           | $-2^{31}$ ( $-2147483648$ ) to $2^{31}-1$ (2147483647)                                                                     | 32-bit signed   |
| unsigned long  | unsigned long int  | 0 to $2^{32}-1$ (4294967295)                                                                                               | 32-bit unsigned |
| float          |                    | Negative range:<br>$-3.4028235E+38$ to $-1.4E-45$<br>Positive range:<br>$1.4E-45$ to $3.4028235E+38$                       | 32-bit IEEE 754 |
| double         |                    | Negative range:<br>$-1.7976931348623157E+308$ to $-4.9E-324$<br>Positive range:<br>$4.9E-324$ to $1.7976931348623157E+308$ | 64-bit IEEE 754 |
| long double    |                    | Negative range:<br>$-1.18E+4932$ to $-3.37E-4932$<br>Positive range:                                                       | 80-bit          |

# Variables and Data Type

- An **integer** is a whole number that doesn't have a fractional part
- **char** stands for character
- A **string** is a sequence of characters that represents text.
- **Double/ Floating point** types: store numbers with decimal points
- **char** stands for character



# Variables and Data Type

- The syntax:

```
datatype variableName;
```

```
#include <iostream>
using namespace std;
int main()
{
    int x = 12;           // integer
    char c = 'A';         // char
    string day = "Monday"; // string
    double d = 5.2;        // Double
    float a = 2.3f;        // Floating
    bool b = true;         // Boolean

    cout << "integer:\t" << x << endl;
    cout << "char:\t\t" << c << endl;
    cout << "string:\t\t" << day << endl;
    cout << "Double:\t\t" << d << endl;
    cout << "Floating:\t" << a << endl;
    cout << "Boolean:\t" << b << endl;
    return 0;
}
```

# Variables and Data Type

- Variables are used to represent values that may be changed in the program

```
1 // Compute the first area
2 radius = 1.0;
3 area = radius * radius * 3.14159;
4 cout << "The area is " << area << " for radius " << radius;
5
6 // Compute the second area
7 radius = 2.0;
8 area = radius * radius * 3.14159;
9 cout << "The area is " << area << " for radius " << radius;
```

|         |          |
|---------|----------|
| radius: | 1.0      |
| area:   | 3.14159  |
| radius: | 2.0      |
| area:   | 12.56636 |

# Variables and Data Type

- data types int and double, short, long, float, char, and bool.
- If variables are of the same type, they can be declared together, as follows:

```
datatype variable1, variable2,..., variablen;
```

- The variables are separated by commas

```
int i, j, k; // Declare i, j, and k as int variables
```

- C++ allows an alternative syntax

```
int i(1), j(2);
```

```
int i = 1, j = 2;
```

# Variables and Data Type

- Identify and fix the errors in the following code:

```
1  #include<iostream>
2  using namespace std;
3
4  int Main()
5  {
6      int i = k + 1;
7      cout << I << endl;
8
9      int i = 1;
10     cout << i << endl;
11
12     return 0;
13 }
```

# Operations

|                   | Operator              | Type                |
|-------------------|-----------------------|---------------------|
| Unary Operator →  | ++, --                | Unary Operator      |
| Binary Operator { | +, -, *, /, %         | Arithmetic Operator |
|                   | <, <=, >=, ==, !=     | Relational Operator |
|                   | &&,   , !             | Logical Operator    |
|                   | &,  , <<, >>, ~, ^    | Bitwise Operator    |
|                   | =, +=, -=, *=, /=, %= | Assignment Operator |

# Assignment Statements and Assignment Expressions

- An assignment statement **designates a value for a variable.**
- After a variable is declared, you can assign a value to it by using an assignment statement.
- the equal sign (=) is used as the assignment operator.
- The syntax for assignment statements is as follows:

```
variable = expression;
```

# Assignment Statements and Assignment Expressions

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      // Syntax: variable = expression;
6      // 1. Simple Assignment
7      int x1 = 10; // assigns 10 to x
8
9      // 2. Assignment as Expression
10     int a, b;
11     a = (b = 5) + 2; // b is assigned 5, then a is assigned (5 + 2) = 7
12
13     // 3. Chained Assignment
14     int x, y, z;
15     x = y = z = 15;
16
17     // 4. Combined Assignment Operators
18     int num = 10;
19     num += 5; // same as num = num + 5; -> num becomes 15
20     num -= 3; // num = num - 3; -> num becomes 12
21     num *= 2; // num = num * 2; -> num becomes 24
22     num /= 4; // num = num / 4; -> num becomes 6
23     num %= 5; // num = num % 5; -> num becomes 1
24
25     return 0;
26 }
27
```



# Named Constants

- A named constant is an identifier that represents a permanent value.

```
const datatype CONSTANTNAME = value;
```

- constants are named in uppercase: **PI**, not **pi** or **Pi**

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      const double PI = 3.14159;
7
8      // Step 1: Read in radius
9      double radius;
10     cout << "Enter a radius: ";
11     cin >> radius;
12
13     // Step 2: Compute area
14     double area = radius * radius * PI;
15
16     // Step 3: Display the area
17     cout << "The area is ";
18     cout << area << endl;
19
20     return 0;
21 }
```



# W3 - Practice

## Exercise 1.

- Write a program that reads a Celsius degree in a double value from the console, then converts it to Fahrenheit and displays the result. The formula for the conversion is as follows:

```
fahrenheit = (9 / 5) * celsius + 32
```

*Hint:* In C++, **9 / 5** is **1**, but **9.0 / 5** is **1.8**.

- Here is a sample run:

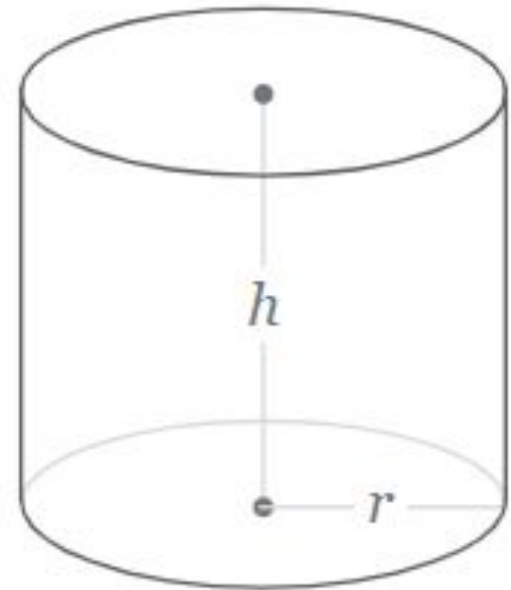
```
Enter a degree in Celsius: 43 
43 Celsius is 109.4 Fahrenheit
```

## Exercise 2.

- Write a program that reads in the radius and length of a cylinder and computes the area and volume using the following formulas:

```
area = radius * radius *  $\pi$   
volume = area * length
```

- Here is a sample run:



```
Enter the radius and length of a cylinder: 5.5 12 ↵ Enter  
The area is 95.0331  
The volume is 1140.4
```

## Exercise 3.

- Write a program that reads a number in feet, converts it to meters, and displays the result. One foot is 0.305 meter.
- Here is a sample run:

```
Enter a value for feet: 16.5 ↵ Enter  
16.5 feet is 5.0325 meters
```

Thanks!